Relational analyses with a super super fast solver

Eris

October 20, 2026

1 stuff

Let I be some set of values, perhaps integers modulo 2^{64}. Fix a sub-monoid M of $\mathrm{End}_{\mathbf{Set}}(I)$. Take an M-labelled digraph (V, E). This would be constructed by the algorithm and describes how variables relate to each other. Take a set of valid states $S \subseteq (V \rightharpoonup I)$, which are partial maps from vertices (variables) to values. This could be the lfp of some ssa semantic operator or something like that. Impose the condition that if the set of out-edges of v is nonempty, then for all states $s \in S$ where v is defined, there is some $(f, v') \in \mathrm{out}(v)$ such that $s(v) = f(s(v'))$. Note that we can compose labels of edges in a path to get a label of a path via monoid multiplication.

If there are no edges out of a vertex, we can modify the graph so it has a self-loop labelled with the identity. Then the imposed condition holds without any presupposition!

Lemma 1.1. For every natural number $n \in \mathbb{N}$ and every state $s \in S$ where v is defined, there is a path p of length n such that for all sub-paths $(f, v') \leq p$, $s(v) = f(s(v'))$.

Proof. The base case of $n = 0$ is trivially satisfied, as the only sub-path of the path of length 0 is (id, v).

Suppose the proposition holds for n, and we have a path (f, v') of length n for a given state s. Then by assumption, there is some edge (g, v'') from v' such that $s(v') = g(s(v''))$. Then the path of length $n + 1$, $(g, v'') \circ (f, v')$ satisfies the condition. □

Lemma 1.2. If v' dominates v in the CFG, every terminating path from v contains v', and all paths to v' have the same label f, then for all states $s \in S$ where v is defined, $s(v) = f(s(v'))$.

Proof. Unprovable without more assumptions and definitions!! But I'm sure it's true... □

2 Old stuff

This explanation needs lots more pictures and lots more theory which will be done at another time.

The IDE framework described in [SRH96] lets us perform interprocedural analyses efficiently given the domain we are analysing with satisfies nice enough properties (namely, distributy and efficient edge function representation). An example analysis that the framework lets us perform is linear constant propagation, which is constant propagation through expressions that are single variable linear functions. A natural question that someone familiar with the framework may ask is, could this be generalised to an interprocedural linear expression propagation analysis (generalising copy propagation)? The originally presented IDE solver likely does not give us a way to obtain

these results, as the phase two analysis results it gives us will be non relational. However, if we manually try and work through a linear expression propagation analysis ourselves, an IDE-like approach emerges.

Assuming an SSA IR, we can construct a graph where each variable is a node. Because of the SSA property, each variable will be assigned a unique expression, and if this expression is a linear function of another variable, we could draw a (directed) edge function between the variables. Repeating this for every variable in the program, we obtain a graph where a variable may point to, through a chain, some source variable, and hence can be replaced with a linear function (given by the composite of the chain) of said source variable. To make this efficient we can apply a labelled union find data structure (labels being the edges). This gives us a working analysis but it can be made more precise by handling branching and by handling interprocedural calls.

If a variable is assigned a phi of two other variables, but those two variables are really equal to each other, then this new variable is itself also equal. We can encode phis in our graph as having multiple edges coming out of one variable. Then, once all edges are added to the graph, we can perform a recursive phi collapsing process where we look at the targets of each parent variable, collapse those targets, then check to see whether all targets have the same parent with the same function. If so, we can set our parent to that parent with the new composed function. There are weird whacky things that need to be done to handle cycles but it isn't too bad.

If some collection of variables are returned by a function call, it could be the case that those output variables are linear functions of a collection of input variables, and so it could be possible that new edges could be added to our graph. We can check the body of the callee and see whether, after doing a path compressed dfs, any of the output variables have their phi-composed parents all being input variables. If so, we can add the composed edges. This will not give complete information if the callee isn't itself fully analysed. To handle this, we can take the standard approach of computing a reverse topological ordering of the strongly connected components of the call graph using perhaps Tarjan's algorithm. Then any calls to out-of-scc procedures will have complete information. Else, if we have a (possibly mutually) recursive call, we should notify call callers of the currently being analysed procedure that there has been an update to its graph every time a call propagates information. It could be assumed that all procedures in the scc have had their assign and phi edges already added so that those do not need to propagate information.

Complexity analysis sounds hard!

References

[SRH96] Mooly Sagiv, Thomas Reps, and Susan Horwitz. Precise interprocedural dataflow analysis with applications to constant propagation. *Theoretical Computer Science*, 167(1):131–170, January 1996.